

Assignment #3 - IPC

System Programming (Spring, 2026) — Jun 3, 2026

Background

프로세스 간 통신

프로세스 간 통신(Inter-Process Communication, **IPC**)이란 프로세스들 사이에 서로 데이터를 주고받는 행위 또는 그에 대한 방법이나 경로를 뜻한다.

IPC는 마이크로커널과 나노커널의 디자인 프로세스에 매우 중요하다: 마이크로커널은 커널이 제공하는 기능의 수를 줄여준다. 해당 기능들은 IPC를 통해 서버와 통신함으로써 얻으며 일반적인 모놀리식 커널에 비해 IPC의 수가 극적으로 증가된다.

ko: https://ko.wikipedia.org/wiki/프로세스_간_통신

en: https://en.wikipedia.org/wiki/Inter-process_communication

Goal

이번 과제의 목표는 **간단한 IPC 드라이버**와 **user-space client/server 프로그램**을 구현하면서, IPC(Inter-Process Communication)가 어떻게 동작하는지 이해하는 것입니다.

Simple-IPC Kernel Module

이번 과제에서 구현할 Simple-IPC 커널모듈의 실행 흐름은 다음과 같습니다.

- 클라이언트에서 **PUT** 명령으로 Key-Value 쌍을 서버에 전달한다.
- 서버는 전달받은 Key-Value 쌍을 내부에 저장한다.
- 클라이언트가 **GET** 명령으로 특정 Key를 요청하면, 서버는 해당 Key에 대응하는 Value를 반환한다.

```
$ ./client PUT mykey 100
$ ./client GET mykey
100
```

이 일련의 과정은 매우 간단하지만, 실제로 이를 동작시키기 위해서는 user-space process와 kernel-space 사이에서 다양한 IPC 메커니즘이 함께 동작해야 합니다. 이를 단계별로 나누면 다음과 같습니다.

STEP 1. 커널 모듈 로드 및 서버 프로세스 준비

- IPC 커널 모듈을 로드한다.
- 서버 프로세스를 커널 모듈에 등록(bind)한다.
 - 이 과정에서 커널이 서버를 고유한 IPC endpoint로 식별할 수 있도록 문자열 기반 식별자를 전달해야 한다.
 - 예를 들어, socket 기반 IPC에서는 서버 소켓을 특정 포트 번호에 bind하여 endpoint를 식별한다.

STEP 2. 클라이언트의 IPC 요청 수행

- 클라이언트는 **ioctl()** 시스템 콜을 통해 커널 모듈에 IPC 요청을 전달한다.
- 커널 모듈은 전달받은 IPC 메시지를 적절한 서버 endpoint로 전달한다.
- 서버 프로세스는 요청을 처리한 뒤, 결과를 다시 커널 모듈을 통해 클라이언트에 반환한다.
- 클라이언트는 필요한 경우 결과를 터미널에 출력한다.

Requirements

본 과제에서는 다음 요구사항을 만족하는 프로그램을 작성하여 제출합니다.

1. Kernel Module (LKM)

- 커널 모듈은 IPC 메시지를 중개하는 기능만 포함해야 합니다. 클라이언트와 서버 간 통신에서 수행되는 application-level logic은 커널 모듈에 의존해서는 안 됩니다.
 - 예를 들어, `PUT`, `GET` 과 같은 명령의 의미 해석 및 Key-Value 저장/조회 로직은 user-space server에서 수행해야 합니다.
- IPC는 동기식으로 처리되어야 합니다.
 - 클라이언트가 커널 모듈에 `ioctl()` 을 호출하여 요청을 전달하면, 서버가 해당 요청을 처리하고 응답을 반환할 때까지 클라이언트 프로세스는 blocking 상태가 되어야 합니다.
 - 이를 구현하기 위해 커널 모듈 내부에서 `wait_event_interruptible()` 및 `wake_up_interruptible()` 을 사용할 수 있습니다.
- user-space와 kernel-space 간 데이터 전달에는 `copy_to_user()` 및 `copy_from_user()` 함수를 활용하세요.
 - user-space 포인터를 직접 참조(dereference)하는 방식은 사용하지 않습니다.
- 커널 모듈은 서버가 bind될 때, 커널 모듈은 다음 형식의 커널 로그를 출력해야 합니다.

```
# 식별자 KV_STORE인 서버가 바인딩을 하는 경우
BIND KV_STORE

# 식별자 KV_STORE인 서버가 언바인딩을 하는 경우
UNBIND KV_STORE
```

2. Server

- 서버는 커널 모듈과 bind할 때 사용할 식별자를 명령 인수(command-line argument)로 전달받아야 합니다.

```
$ ./server KV_STORE
```

- 서버는 전달받은 식별자를 이용하여 커널 모듈에 자신을 bind해야 합니다.
- 서버는 종료 시 반드시 커널 모듈로부터 자신을 unbind 해야 합니다.
 - 정상 종료뿐만 아니라 인터럽트 시그널(Ctrl+C) 등에 의한 종료 상황도 고려해야 합니다.
 - 서버가 종료된 이후에도 커널 모듈이 해당 서버를 여전히 등록된 상태로 인식해서는 안 됩니다.
 - 이를 구현하기 위해 `signal()`, `sigaction()`, `atexit()` 등의 함수를 활용할 수 있습니다.
- 서버는 클라이언트 요청을 처리하기 위한 별도의 Key-Value Table을 유지해야 합니다.
 - Key와 Value의 자료형은 문자열(string)로 제한합니다.
- 서버 프로그램은 `PUT` 요청과 `GET` 요청을 처리할 때 다음 형식의 로그를 출력해야 합니다.

```
# Key: mykey, Value: 100의 PUT 명령이 들어온 경우
PUT K: mykey V: 100

# Key: mykey, Value: 100의 GET 명령이 들어온 경우
GET K: mykey V: 100
```

3. Client

- 클라이언트는 사용자가 실행할 명령과 엔드포인트 식별자를 명령 인수로 받습니다.
- 클라이언트는 전달받은 명령을 IPC 메시지 형태로 구성한 뒤, `ioctl()` 시스템 콜을 통해 커널 모듈에 전달해야 합니다.

```
# Usage:
# ./client <endpoint> PUT <key> <value>
# ./client <endpoint> GET <key>
# PUT 명령
$ ./client KV_STORE PUT mykey 100

# GET 명령
$ ./client KV_STORE GET mykey
100
```

기타 요구사항

- Device name은 채점의 일관성을 위해 반드시 `simipc` 를 사용합니다.
- 본 과제는 반드시 VM 또는 별도의 Linux 환경에서 수행하여야 합니다.
 - 개발 환경 구성 방법은 이전에 Bb에 게시된 **LKM Tutorial** 자료를 참고하시기 바랍니다.
- 반드시 주석을 포함하여 제출해야 합니다.
 - 주석은 프로그램의 ABI(Application Binary Interface)가 되는 헤더 파일, 주요 자료구조, ioctl command, 그리고 주요 함수에 작성하는 것을 권장합니다.
 - 참고: <https://leffept.tistory.com/414>
- 필요한 경우 `README.md` 파일을 작성할 수 있습니다.
 - `README.md`에는 전체 프로젝트의 빌드 방법, 커널 모듈 로드/언로드 방법, 서버 및 클라이언트 실행 방법, 테스트 예시 등을 포함할 수 있습니다.

Submission

본 과제는 다음 제출 기준에 맞추어 제출해야 합니다.

- **제출 파일:** 전체 프로젝트 압축 파일
 - user-space application인 client, server 프로그램과 kernel module source code를 모두 포함해야 합니다.
 - 빌드 자동화를 위한 Makefile 또한 반드시 포함해야 합니다.
 - 압축 파일 이름은 학번으로 설정하여 제출합니다.
 - 예: 202224210.zip
- **제출 기한: 6월 16일 (화) 11:59 PM**

TA Comment

프로젝트 구성에 어려움을 겪는 경우, 과제와 함께 제공되는 skeleton project를 활용할 수 있습니다.

해당 프로젝트에는 기본적인 디렉터리 구조, 예제 코드, 그리고 빌드 자동화 스크립트(Makefile)가 포함되어 있습니다.

Grading Criteria

본 과제는 총 15점으로 구성되며, 기본적으로 테스트 케이스 통과 여부를 기준으로 채점합니다. 이후 기타 요구사항을 만족하지 못한 경우 감점을 적용합니다.

- **테스트 케이스 통과 (15)**
 - 임의의 서버 식별자 및 Key-Value 쌍을 이용하여 테스트를 진행합니다.
 - 테스트 케이스 세트는 공개하지 않습니다.
 - 동일한 Key에 대해 기존 Value를 overwrite하는 상황은 테스트 범위에 포함하지 않습니다.
 - `PUT` 기능만 정상 동작하고 `GET` 기능이 동작하지 않는 경우, 테스트 케이스 점수의 최대 50%까지만 득점할 수 있습니다.
- **감점 사항**
 - 커널 로그가 요구사항에 맞게 출력되지 않는 경우 (-3점)
 - 서버 로그가 요구사항에 맞게 출력되지 않는 경우 (-3점)
 - 서버 종료 시 unbind가 수행되지 않는 경우 (-3점)

기타 감점 사항

- **Plagiarism**
 - 치팅(카피)이 심각하게 의심되는 경우, 예외 없이 0점 처리합니다.
 - 단, 과제 수행 과정에서 학습 보조 목적으로 생성형 AI(GPT, Gemini 등)를 활용하는 것은 허용합니다.
- 제출 형식이 올바르지 않은 경우: -1점
- 지각 제출:
 - 기본 -1점
 - 이후 추가로 시간당 -1점
- 주석이 전혀 없는 경우: 예외 없이 0점